CryptoChain Analyzer Dashboard

Final Project Report -- Hash Functions and Blockchain

Cryptography | Universidad Alfonso X el Sabio | Prof. Jorge Calvo | AY 2025-26

Enrique Ruiz Santos . esantrui . May 2026

1 Introduction

This report documents the CryptoChain Analyzer Dashboard, a real-time Python dashboard built with Streamlit that connects to public Bitcoin blockchain APIs and displays live cryptographic metrics. The project was developed as the individual assessment for the Cryptography course at Universidad Alfonso X el Sabio (AY 2025-26), under the supervision of Prof. Jorge Calvo.

The dashboard implements all four required modules (M1-M4). Data is sourced from three free public APIs: Blockstream, Blockchain.info, and the Blockchain.info Charts API. The page auto-refreshes every 60 seconds so all metrics stay current without manual intervention.

2 Cryptographic Metrics

2.1 SHA-256d and Proof of Work (M1, M2)

SHA-256d (double SHA-256) is Bitcoin's standard cryptographic hash: SHA256(SHA256(data)). Bitcoin's Proof-of-Work requires miners to find a nonce such that SHA256d(header) <= target. The 80-byte block header contains six fields, all stored in little-endian byte order:

- version (4 B) -- signals which BIP consensus rules apply
- previous block hash (32 B) -- SHA-256d of the preceding header; the chain link
- Merkle root (32 B) -- root of the transaction hash tree
- timestamp (4 B) -- Unix epoch in seconds
- bits (4 B) -- compact encoding of the 256-bit PoW target threshold
- nonce (4 B) -- miner-controlled counter iterated until the hash is valid

Module M2 parses all six fields and verifies the PoW locally using Python's built-in hashlib and struct modules -- no external cryptographic library is used. The verification proceeds in six explicit steps visible in the dashboard: (1) reconstruct the 80-byte header from the API fields, (2) compute SHA256(header), (3) compute SHA256 again, (4) reverse bytes to display order, (5) compare with the API-returned hash, (6) confirm the result is numerically less than the decoded target.

bits -> target decoding: target = coefficient x 2^(8 x (exponent - 3))
where exponent = bits >> 24 and coefficient = bits & 0x00FFFFFF.
A valid hash, interpreted as a 256-bit unsigned integer, must be <= target.
At current difficulty, approximately 79 leading bits must be zero: P(valid) ~= 2^-79.

Field	Example value	Meaning
bits (hex)	0x17021FF0	Compact target stored in block header
Exponent byte	0x17 = 23	target = coeff x 2^(8x(23-3)) = coeff x 2^160
Coefficient	0x021FF0	Mantissa of the compact representation
Full 256-bit target	00000000021FF0...00	Hash must be <= this value
Leading zero bits	~79 / 256	P(valid hash) ~= 2^-79

2.2 Difficulty and Network Hash Rate (M1, M3)

The difficulty value is derived from the target by comparing it to the genesis block target (bits = 0x1d00ffff):

```
difficulty = genesis_target / current_target
```

The estimated network hash rate follows from the expected hashes per block and the 600-second target:

```
hashrate ~= difficulty x 2^32 / 600   [H/s]
```

At current difficulty (~1.1 x 10^14) this yields approximately 790 EH/s (exahashes per second), consistent with public mining pool statistics.

Bitcoin adjusts difficulty every 2,016 blocks (~14 days):

```
new_difficulty = old_difficulty x (actual_time_for_2016_blocks / 1,209,600 s)
```

The ratio is clamped to [1/4, 4] to prevent extreme swings. Module M3 charts this history, marks each adjustment event with a diamond marker, and plots the block-time ratio per epoch derived analytically from consecutive difficulty values: actual_block_time / 600 = D_old / D_new.

2.3 Inter-Block Time Distribution (M1)

Module M1 fetches timestamps for the last 50 blocks via the Blockstream API and plots the distribution of inter-block arrival times. The expected distribution is Exponential(lambda = 1/600) because:

- Each SHA-256 hash attempt is an independent Bernoulli trial with probability p ~= 1 / (difficulty x 2^32) of success.
- The number of attempts until success is Geometric(p), which for large n and small p converges to an Exponential distribution.
- The network self-adjusts so the mean inter-arrival time stays at 10 minutes.

The histogram is overlaid with both the theoretical Exp(lambda = 1/10 min) curve and an empirical fit, confirming the Poisson-process nature of Bitcoin mining.

3 AI Component -- M4: Difficulty Predictor

3.1 Problem Statement

The goal of Module M4 is to predict the next Bitcoin mining difficulty adjustment value using a time-series model trained on historical difficulty data fetched from the Blockchain.info Charts API. An accurate forecast helps miners and analysts anticipate profitability changes before the next 2,016-block epoch closes.

3.2 Model Choice and Justification

Four models are implemented and compared on a holdout set (last 20 % of data, no shuffle):

Model	Rationale	Key parameter
ARIMA(1,2,1)	Double-differencing removes Bitcoin's strong upward trend making the series stationary. AR(1) captures first-order autocorrelation.	d = 2 (integrated order)
SARIMA(0,2,1)x(0,1,1,7)	Extends ARIMA with a seasonal component (period = 7 days) to capture weekly mining pool patterns.	Seasonal period s = 7
Holt-Winters	Explicit triple-exponential smoothing (additive trend + additive seasonality). Robust for medium-horizon forecasts.	seasonal_periods = 7
Linear Regression	Global trend baseline -- fits a straight line through the time index. Interpretable and computationally cheap.	deg = 1 (OLS)
Ensemble	Simple average of ARIMA, Holt-Winters, and Linear Regression. Reduces model-specific variance; recommended for general use.	Equal weights (1/3 each)

Why time-series models over deep learning? LSTM and Prophet require substantially more data and longer training time. The difficulty series, sampled approximately daily, provides ~180 training points -- sufficient for ARIMA/ETS-class models but borderline for neural approaches. The ensemble strategy further reduces the risk of any single model's bias dominating the forecast.

3.3 Evaluation Results

Each model is trained on the first 80 % of data and evaluated on a 30-day holdout set. Metrics are computed from real blockchain.info difficulty data (180 data points, difficulty range: 1.26e14 to 1.55e14). Confidence intervals shown in the dashboard are +/-1 sigma of in-sample residuals, expanding linearly with the forecast horizon.

Model	MAE	RMSE	MAPE (%)	Notes
ARIMA(1,2,1)	1.36e+13	1.60e+13	10.17 %	Overreacts to short-term volatility
SARIMA	~ARIMA	~ARIMA	~10 %	Seasonal term has limited effect on difficulty
Holt-Winters	4.79e+12	5.38e+12	3.59 %	Best single model on this period
Linear Regression	1.32e+12	1.73e+12	0.97 %	Best overall -- difficulty trended linearly over 180 days
Ensemble	5.98e+12	7.01e+12	4.48 %	More robust across different market regimes

Interpretation: Linear Regression achieves the lowest MAPE (0.97 %) on this holdout because Bitcoin difficulty followed an approximately linear upward trend over the last 180 days. The Ensemble model sacrifices some in-sample accuracy for better generalisation when the regime changes (e.g. sudden hash-rate drops). The dashboard presents all five models with their holdout metrics so the user can choose

the most appropriate one.

3.4 Anomaly Detection (M4 -- built-in)

An anomaly detector is integrated into M4. For each data point in the difficulty series, a Z-score is computed:

```
z = (D_i - mean) / std
```

Points with |z| >= threshold (default 2.0) are flagged as anomalies and displayed as red triangles on the forecast chart. In the difficulty context, anomalies correspond to unusually large or small adjustment events -- possible indicators of sudden hash-rate changes, mining pool exits, or coordinated attacks on the network.

4 Technical Implementation

The dashboard is built with Streamlit using a sidebar navigation structure with five pages (Overview + M1-M4). Charts are rendered with Plotly graph_objects. The project has no external cryptographic dependencies: all SHA-256 operations use Python's built-in hashlib and struct modules.

Component	Details
APIs	Blockstream (block data, raw headers, recent timestamps); Blockchain.info (rawblock, latestblock); Blockchain.info Charts (difficulty history, avg-confirmation-time)
Machine Learning	statsmodels ARIMA, SARIMAX, ExponentialSmoothing; numpy polyfit for Linear Regression
Frontend	Streamlit, Plotly graph_objects, custom dark-theme CSS with gradient header
Cryptography	hashlib.sha256 (SHA-256d), struct (little-endian header parsing) -- no external library
Auto-refresh	60-second non-blocking refresh (time.sleep(60) + st.rerun()); M1 uses session-state caching with 60-second TTL
Error handling	All API calls wrapped in try/except; stale cached data used as fallback. M3 stores data in session_state so it persists across auto-refreshes.

5 References

[1] Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf. Cited for §6 (mining incentives), §7 (difficulty recalculation), §11 (attack probability).

[2] Blockstream. Esplora REST API Documentation. https://github.com/Blockstream/esplora/blob/master/API.md. Used for block raw headers and recent block timestamps (M1, M2).

[3] Blockchain.com. Charts API -- Difficulty and Hash Rate. https://www.blockchain.com/explorer/charts. Used for historical difficulty data and average confirmation time (M3, M4).

[4] Seabold, S. & Perktold, J. (2010). statsmodels: Econometric and statistical modeling with Python. Proceedings of the 9th Python in Science Conference. https://www.statsmodels.org. ARIMA/SARIMAX/Holt-Winters implementation (M4).

[5] Bitcoin Wiki. Block hashing algorithm. https://en.bitcoin.it/wiki/Block_hashing_algorithm. Reference for

80-byte header structure and byte-order conventions (M2).